

Paddle Inference Streams

目录

- 背景
 - io stream + exec stream
 - multi exec stream
 - predictor stream
- 需求
- 竞品调研
 - onnxruntime
 - tensorflow
 - pytorch
- 多流方案
 - io与exec多流
 - predictor stream
- 前置依赖工作
- 开发计划及排期

背景

多流此处包括多个概念，以下简单介绍

1. io流和exec（执行）流（依赖项：默认使用pinned memory）
2. 多个exec流（io流可考虑和多个exec流同步使用，复杂场景）
3. 执行器级别的流，每个执行器拥有自己的流
4. 以上概念中，io流可拆分为 cpu->gpu io 流和 gpu->cpu io 流（显卡有d2h copy engine和h2d copy engine）

io stream + exec stream

io流和exec流并发，顾名思义，主要是为了，数据传输和kernel执行能够并发，当业务场景中需要多个predictor时有效。

暂无具体业务场景。

multi exec stream

模型级并发，多个流运行同一个模型，仅当模型并行度高的场景有效。

模型并行，低优考虑。

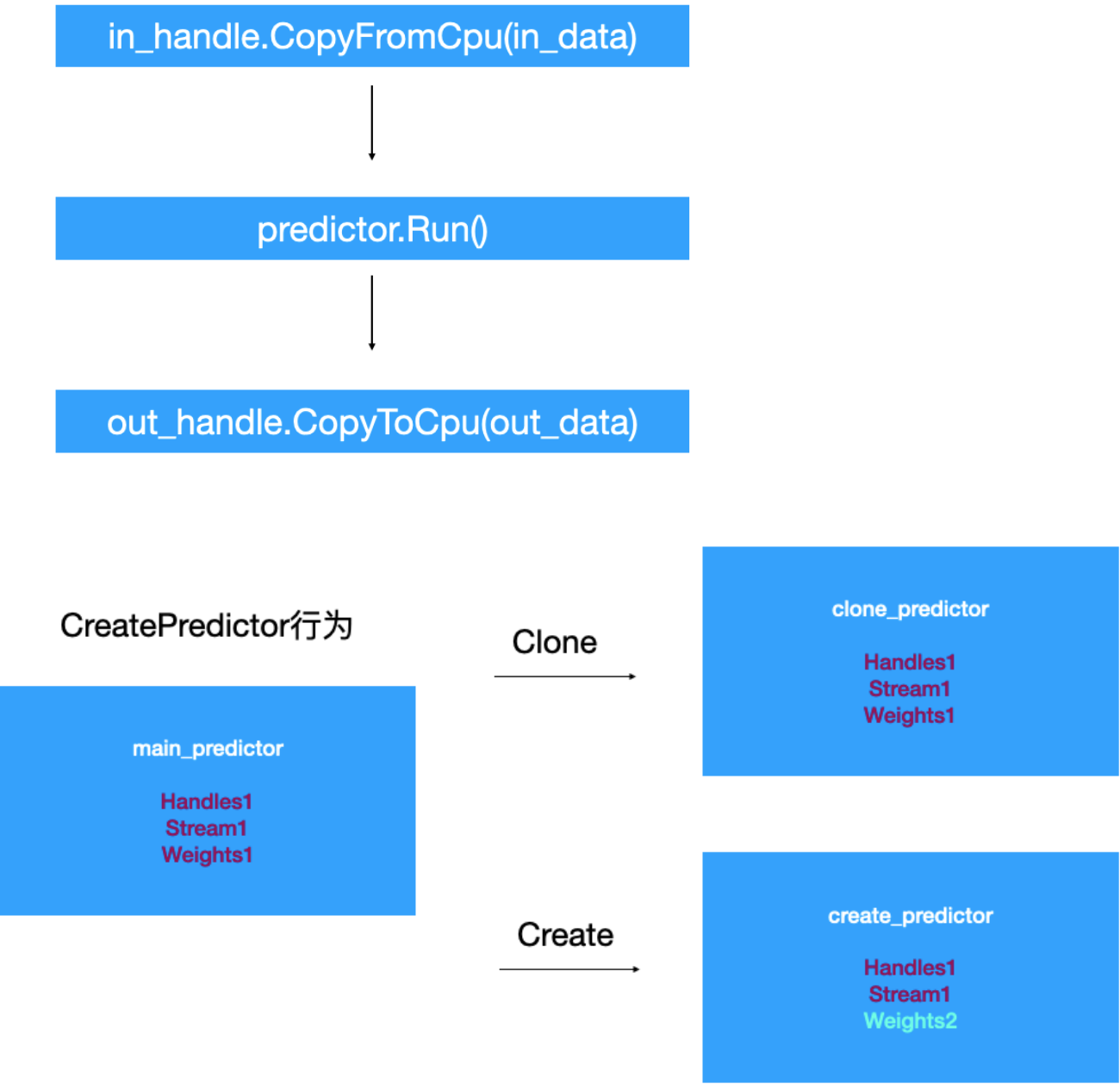
predictor stream

执行器级别流，每个执行器拥有自己的stream，当模型计算所需资源小于gpu资源时有效。

predictor clone处理额外关注。

需求

Paddle Inference现有逻辑如下：



以上逻辑主要存在以下问题：

优点：接口简单，易用性好。

缺点：

- 所有操作均在同一个stream上，难以进行io和exec的并行；
- CopyToCpu的过程会默认Sync，难以做精细的性能优化；
- 无法接收外部用户的stream；
- clone或new得到的Predictor会Share predictor的所有资源。

经过初步分析，主要需求如下：

1. 多stream并发；
2. 外部用户可设置stream， io && exec stream。

竞品调研

onnxruntime

```

/// <summary>
/// Options for the CUDA provider that are passed to SessionOptionsAppendExecutionProvider_CUDA_V2.
/// Please note that this struct is *similar* to OrtCUDAProviderOptions but only to be used internally.
/// Going forward, new cuda provider options are to be supported via this struct and usage of the publicly defined
/// OrtCUDAProviderOptions will be deprecated over time.
/// User can only get the instance of OrtCUDAProviderOptionsV2 via CreateCUDAProviderOptions.
/// </summary>
struct OrtCUDAProviderOptionsV2 {
    int device_id; // cuda device id.
    int has_user_compute_stream; // indicator of user specified CUDA compute stream.
    void* user_compute_stream; // user specified CUDA compute stream.
    int do_copy_in_default_stream; // flag specifying if the default stream is to be used for copying.
    OrtCudnnConvAlgoSearch cudnn_conv_algo_search; // cudnn algo search enum.
    size_t gpu_mem_limit; // BFC Arena memory limit for CUDA.
    onnxruntime::ArenaExtendStrategy arena_extend_strategy; // (will be overridden by contents of `default_memory_arena_cfg` is it exists)
    OrtArenaCfg* default_memory_arena_cfg; // BFC Arena extension strategy.
    int cudnn_conv_use_max_workspace; // (will be overridden by contents of `default_memory_arena_cfg` is it exists)
    int enable_cuda_graph; // BFC Arena config flags.
    int cudnn_conv1d_pad_to_nc1d; // flag specifying if maximum workspace can be used in cudnn conv algo search.
    // flag specifying if the CUDA graph is to be captured for the model.
    // flag specifying if pad Conv1D's input [N,C,D] to [N,C,1,D] or [N,C,D,1].
};
  
```

onnxruntime的cuda provider在stream方面提供了以下能力：

- 支持用户传compute stream
- 支持io stream（测试没起到作用，需进一步阅读源码，onnxruntime默认io流和计算流是同一个流（根据issue和相关修复pr）似乎存在什么问题，没有默认放开，需花时间跟进）

tensorflow

todo

pytorch

todo

多流方案

io与exec多流

缺失业务场景，暂时hold。

（优化方向预处理和后处理逻辑和模型执行的并发）

简化问题，此处只思考一个input流，一个output流和一个exec流的情况。

该问题处理相对比较简单，框架和接口需要满足以下需求：

- ~~io stream与exec stream间插入event同步；~~
- 用户可传入io && exec stream；
- 用户可获取内部io && exec stream；

predictor stream

执行器的流，可以选择**共享**或**私有**两种策略，通过config来配置接口。

```
</> C++ | 收起 ^  
  
1 config.SetStreamStrategy(StreamStrategy::kExternal,  
   StreamStrategy::kShare);  
2 config.SetExecStream(execution_stream);  
3 config.SetStreamStrategy(StreamStrategy::kInternal,  
   StreamStrategy::kPrivate);
```

如果是共享策略，则行为和过去的predictor一致，所有的Predictor共享一份stream和handle资源。

如果是私有策略，则每个执行器拥有独属于自己的stream和handle资源：

- 如果是外部用户设置stream，则predictor clone的时候需要显式传递stream参数；

```
</> C++ | 收起 ^  
  
1 predictor.clone(new_stream);
```

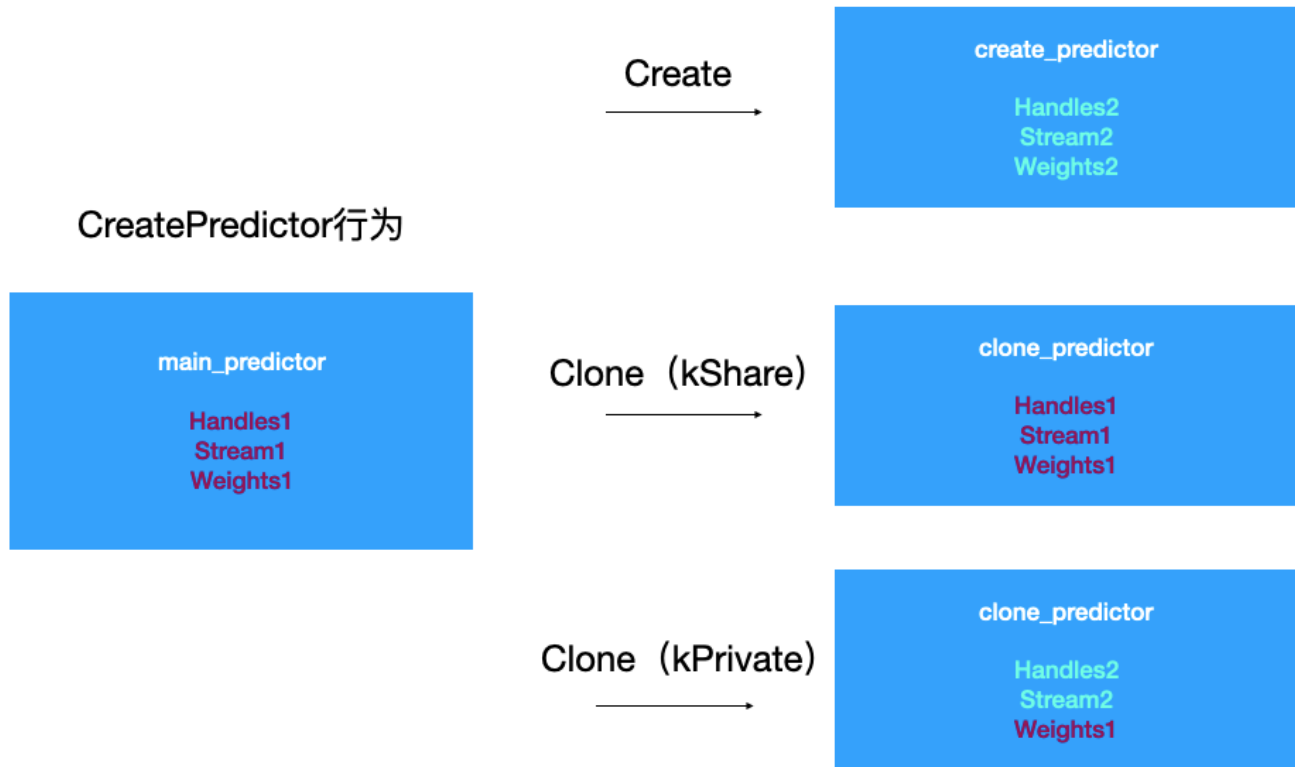
- 如果是使用框架的stream，则clone得到的predictor负责stream及handle资源的管理。（默认行为）

createPredictor的行为是predictor hold handle资源，根据config配置，可选是否hold stream；

clone的行为随StreamStrategy变化：

- StreamStrategy::kShare, clone得到的predictor share handle资源, share stream;
- StreamStrategy::kPrivate, clone得到的predictor hold hanlde资源, hold stream。

行为如下:



前置依赖工作

- 此处仅考虑phi算子库全部迁移完后的场景, 即算子调度仅走phi算子 (废弃OperatorWithKernel体系)。
- 模型输入输出默认设置为pinned memory。

开发计划及排期

1. config增加外部设置exec stream的接口; 0.5人天
2. Predictor增加DeviceContexts成员, 以及ResourceManager类管理; 0.5人天
3. NaiveExecutor Run接收DeviceContexts参数; 0.5人天
4. Operator体系接收DeviceContexts参数重载; 5人天
5. ~~OperatorBase支持device_contexts参数~~
6. ~~OperatorWithKernel支持DeviceContexts参数~~
7. ~~tensorrt engine op需修改, 看如何接收DeviceContexts参数;~~
8. 所有继承operatorWithKernel的算子, 需从runtime context中获取device context; 0.5人天
9. predictor 支持私有化clone, 及config相关接口配置; 2人天
10. 单测测试。3人天

预计时间两周